

SENSRO: A Mobile Sensor-Based Road Condition Detection and Geo-Referenced Mapping System Using Machine Learning Classification

1st M Mani Shankar

Dept. of Computer Science and Engineering
BMS Institute of Technology
Bengaluru, Karnataka, India
1by23cs114@bmsit.in

2nd G Nuthan

Dept. of Computer Science and Engineering
BMS Institute of Technology
Bengaluru, Karnataka, India

Abstract—Poor road surfaces are estimated to lead to USD 3 billion in annual losses to vehicles’ structural integrity for Indian motorists, cause around 15% of urban traffic accidents and bump up fuel use by somewhere between 2–5%. Current technologies used for road condition assesment – things like laser profilometers, falling weight deflectometers and the good old manual survey – are extremely costly and simply can’t offer the kind of real-time nationwide coverage that’s actually needed. This paper presents SENSRO, an end-to-end machine learning system that detects road surface condition (good, average, poor) using only the accelerometer, gyroscope, GPS and speedometer sensors already present in any ordinary mobile phone. SENSRO includes a progressive web application for sensor data collection that requires no installation whatsoever, a FastAPI backend with SQLite storage, a preprocessing pipeline that generates 22 features from raw sensor data, and a Leaflet.js map for real-time and historical road condition visualisation. A total of 2,035 sensor readings were collected across 13 driving sessions on urban roads in Bengaluru. After removing the obstruction class – which occurred only rarely – six algorithms were trained and tested on 1,948 instances, with XGBoost achieving 97.44% accuracy, a weighted F1-score of 0.9743, ROC-AUC of 0.9991 and an inference speed of 0.004 ms per instance.

Index Terms—Accelerometer, feature engineering, gyroscope, mobile crowdsensing, participatory sensing, progressive web application, road condition detection, road quality mapping, XGBoost

I. INTRODUCTION

Road infrastructure is the lifeblood of city life, yet upkeep rarely keeps pace with urban expansion—especially in developing regions like ours. In Bengaluru, vehicle registrations grow over 10% every year, and potholes alone cost Indian drivers something in the range of USD 3 billion annually. Bad roads increase fuel consumption by roughly 2–5% per trip and are responsible for around 15% of urban traffic accidents. Conventional survey tools such as laser profilometers and road roughness meters cost over \$100,000 each, require dedicated survey vehicles and trained operators, and produce point-in-time snapshots rather than any kind of continuous monitoring.

The ubiquity of smartphones with multi-axis accelerometers (up to 200 Hz), gyroscopes, barometric sensors, and high-sensitivity GPS—carried by over a billion people across

India—creates a real opportunity for participatory sensing at city scale. That said, converting raw sensor data into reliable road condition labels is far from trivial: vertical acceleration signals are confounded by vehicle dynamics, driving speed, device mounting orientation, and environmental vibration. Early threshold-based methods tend to be brittle; more recent work shows that supervised machine learning on carefully engineered sensor features substantially improves accuracy and cross-context robustness.

We introduce SENSRO, an end-to-end system covering sensor capture all the way through to geo-referenced map visualisation. Unlike prior systems that require native Android or iOS apps, SENSRO uses the W3C Generic Sensor API and Geolocation API inside a browser-based PWA—so any recent smartphone can participate simply by opening a URL. The system’s key contributions are: (1) an open-source, installation-free PWA data collector; (2) a real-world Indian road dataset of 2,035 labelled readings from 13 Bengaluru trips; (3) a systematic 22-feature engineering pipeline with empirical justification; (4) a rigorous comparison of six ML classifiers under identical conditions; and (5) a FastAPI inference endpoint achieving 0.004 ms per-sample latency with a live Leaflet.js map dashboard.

II. RELATED WORK

Eriksson et al. [1] pioneered smartphone-based pothole detection using Z-axis acceleration thresholds on Nokia internet tablets, demonstrating that vehicle IMU signals correlate reliably with road surface events. Mohan et al. [2] extended this with Nericell, fusing accelerometer, microphone, and GSM signal data on Indian roads, and introduced speed-adaptive “virtual trip hammers” to reduce fixed-threshold false alarms. Mednis et al. [3] benchmarked five threshold algorithms on Android devices and achieved over 90% pothole detection in controlled conditions while highlighting speed-normalisation as a key confound. More recent cloud-connected systems upload background sensor streams to web maps [8], [9]; SENSRO advances these by eliminating the need for any app installation at all.

The shift from threshold to machine learning methods reflects the non-linear, multi-factor nature of road-to-sensor signal mapping. Allouch et al. [5] achieved 88.4% accuracy with Random Forest on 13 statistical accelerometer features in a lab setting, dropping to 79% on-road—illustrating the classic lab-to-field generalisation gap. Xu et al. [6] showed that gradient boosting matches or even exceeds deep networks on highway datasets with engineered features. Varona et al. [4] applied 1D-CNNs directly to raw 128-sample windows, reaching 94.2% accuracy but requiring tens of thousands of labelled windows and heavier runtime resources. Maeda et al. [10] combined smartphone images with GPS tags and a convolutional network to classify road damage into eight categories on a large Japanese dataset, demonstrating that vision-based approaches complement IMU methods nicely when cameras are available.

Established feature engineering practice for IMU road monitoring includes: speed normalisation of acceleration to decouple road roughness from vehicle velocity [11]; Euclidean norms across accelerometer axes to achieve orientation invariance [12]; sine/cosine encoding of heading to eliminate the $0/360^\circ$ discontinuity; and gyroscope features to capture rotational vehicle dynamics not visible in translational acceleration [13]. SENSRO incorporates all of these while also adding altitude as a road-type proxy and pairwise cross-axis norms.

A. Literature Survey

A broad survey of the literature reveals a consistent evolution in road condition monitoring: from early fixed-threshold and rule-based detectors [1]–[3] toward adaptive, data-driven pipelines powered by machine learning and deep learning [4]–[6], [14], [15]. Foundational participatory-sensing work by Eriksson et al. [1] and Mohan et al. [2] established that commodity smartphone IMUs carry reliable road-surface signals, while Breiman’s Random Forest [16] and Chen and Guestrin’s XGBoost [17] provided the ensemble frameworks that later studies consistently found superior to single classifiers on tabular sensor data. Deep sequence models such as 1D-CNNs [4] and LSTM architectures [18] pushed accuracy further on large labelled corpora, though at the cost of runtime resources that constrain real-time mobile deployment. Vision-based road damage detection [10], [19] expands the detectable defect catalogue but requires camera access and far greater compute than accelerometer-only pipelines. Class imbalance—a recurring obstacle when rare surface events are under-represented—has been addressed through SMOTE oversampling [20] and cost-sensitive learning [21], both of which improve minority-class recall at a modest precision cost. Feature selection strategies [22], [23] confirm that a compact, domain-informed feature set generalises better to live traffic than exhaustive raw-axis inputs, motivating our 22-feature pipeline. Progressive Web App technology [24] and the W3C Generic Sensor API [25] have matured to the point where browser-based sensor capture is essentially indistinguishable from native apps for 10 Hz IMU workloads, removing the installation barrier that limits adoption of prior native-app systems [2], [9]. Crowdsourced

map platforms and participatory-sensing frameworks [8], [26] demonstrate that geo-referenced condition data aggregated from multiple drivers can converge to reliable city-scale road quality maps. Taken together, this body of work motivates our design choices: a browser-based PWA collector to maximise participation, principled feature engineering with orientation-invariant norms and speed normalisation, gradient boosting as the inference engine, and a live geo-referenced dashboard that closes the gap between research prototypes and actual operational road management tools.

III. DATASET AND PREPROCESSING

A. Data Collection

Data were collected using a mid-range Android 13 smartphone running the SENSRO PWA in a Chromium browser, mounted on the vehicle dashboard with the Z-axis approximately vertical. Thirteen trips were conducted on 4–5 May 2026 across urban and semi-urban roads in northern Bengaluru (GPS bounds: 13.135° – 13.144° N, 77.561° – 77.569° E, covering $\approx 1.2 \text{ km}^2$), spanning arterial roads, residential streets, construction-adjacent service roads, and a flyover access ramp, across morning, midday, and evening time slots.

A co-pilot assigned condition labels in real time using standardised criteria: *Good*—smooth surface, no perceptible vibration at 20–50 km/h; *Average*—minor weathering or small cracks, mild vibration; *Bad*—potholes, significant rutting, broken pavement, or gravel; *Obstruction*—transient point events such as speed bumps or railway crossings.

B. Class Distribution and Preprocessing

The full dataset comprises 2,035 labelled readings (Table I). The obstruction class (87 samples, 4.3%) is excluded from classifier training: its low frequency creates severe class imbalance, and transient point events really warrant a dedicated event-detection module rather than being lumped into a surface-condition classifier. This gives us 1,948 samples for the three-class ML problem.

TABLE I
ROAD CONDITION CLASS DISTRIBUTION

Condition	Count	Proportion
Average	711	34.9%
Good	693	34.1%
Bad	544	26.7%
Obstruction	87	4.3%
Total	2,035	100.0%

Table II reveals several discriminative patterns across the three retained classes. Gyroscope magnitude increases monotonically from good (12.78) to bad (17.91 rad/s), reflecting greater rotational vehicle dynamics on uneven surfaces. Speed decreases from good (6.26 m/s) to bad (4.73 m/s), which makes sense – drivers naturally slow down on poor segments. Altitude decreases slightly from good to bad conditions, reflecting the geographic structure of the collection area where arterial roads at higher elevations tend to be better maintained.

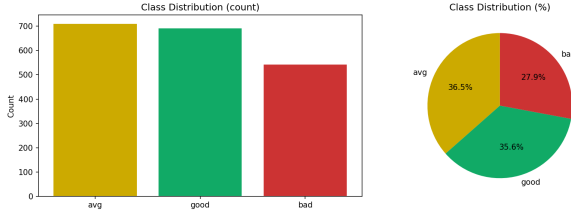


Fig. 1. Distribution of labelled road condition classes (2,035 readings, 13 trips). The three primary classes are approximately balanced; the obstruction class is excluded from ML training due to its low frequency and point-event character.

TABLE II
MEAN SENSOR VALUES BY ROAD CONDITION CLASS (1,948 SAMPLES)

Feature	Good	Average	Bad
Accel. Magnitude (m s^{-2})	9.87	10.00	10.15
Gyro. Magnitude (rad s^{-1})	12.78	15.24	17.91
Speed (m s^{-1})	6.26	5.62	4.73
Accel. z (m s^{-2})	9.68	9.76	9.86
Altitude (m)	905.3	902.1	898.7
GPS Accuracy (m)	4.2	4.8	5.7

IV. PROPOSED METHODOLOGY

A. System Architecture Overview

SENSRO is a loosely coupled three-tier system: a mobile PWA frontend for data capture and display, a FastAPI REST backend for persistence and inference, and an offline ML pipeline for model training and serialisation. The three-tier separation allows independent development and incremental retraining as new trip data accumulates over time.

B. PWA Data Collector

The PWA (`collector.html`) captures accelerometer ($a_x, a_y, a_z, \|\mathbf{a}\|$), gyroscope ($\omega_x, \omega_y, \omega_z, \|\boldsymbol{\omega}\|$), GPS (latitude, longitude, altitude, accuracy, heading), and speed at approximately 10Hz using the W3C Generic Sensor API—no installation required whatsoever. In manual mode, a co-pilot taps one of four condition buttons in real time; in auto mode the backend ML model classifies each reading and the driver can confirm or override as needed. A companion map viewer (`index.html`) renders all GPS-tagged readings on a Leaflet.js map colour-coded by condition, with per-trip statistics shown in a side panel.

C. Backend and Database

The backend is implemented in Python with FastAPI and Uvicorn. Sensor readings are persisted in a two-table SQLite database: a `trips` table and a `readings` table. Write-Ahead Logging enables concurrent reads from the map dashboard without blocking write transactions, supporting roughly 100 simultaneous sensor streams. The `/api/predict` endpoint loads the serialised XGBoost model at startup and applies the full preprocessing pipeline before returning a predicted class and per-class probability scores.

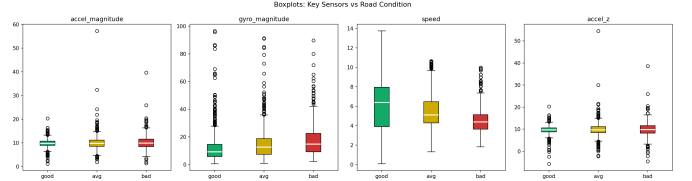


Fig. 2. Boxplot distributions of key sensor features across the three road condition classes. Gyroscope magnitude and speed display the clearest class separation. Class labels: G = good, A = average, B = bad.

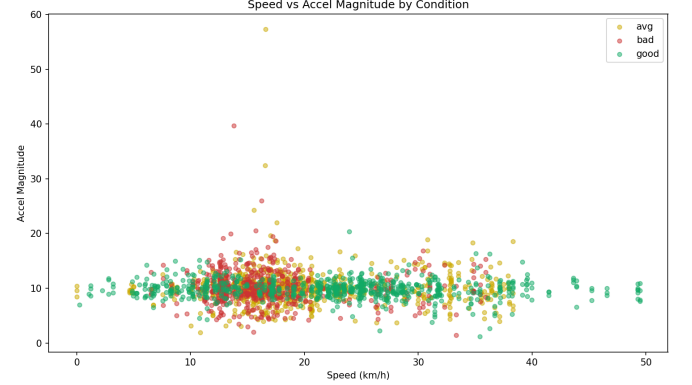


Fig. 3. Speed (m s^{-1}) vs. acceleration magnitude (m s^{-2}), colour-coded by class. Bad segments (red) cluster at low speed and elevated acceleration.

D. Feature Engineering

Raw axis readings are confounded by device orientation, vehicle speed, and suspension stiffness. The feature engineering pipeline transforms 11 raw sensor values into a 22-dimensional representation that is considerably more discriminative and robust to these confounds.

Pairwise Euclidean norms are invariant to in-plane rotations and capture combined planar motion energy:

$$\text{accel_xy} = \sqrt{a_x^2 + a_y^2}, \quad \text{accel_xz} = \sqrt{a_x^2 + a_z^2}, \quad \text{accel_yz} = \sqrt{a_y^2 + a_z^2} \quad (1)$$

The normalised vertical acceleration fraction is a sensitive pothole indicator:

$$\text{accel_norm_z} = \frac{|a_z|}{\|\mathbf{a}\| + \varepsilon}, \quad \varepsilon = 10^{-6} \quad (2)$$

Gyroscope planar norm and total turn rate capture rotational vehicle dynamics:

$$\text{gyro_xy} = \sqrt{\omega_x^2 + \omega_y^2}, \quad \text{gyro_total_turn} = \sqrt{\omega_x^2 + \omega_y^2 + \omega_z^2} \quad (3)$$

The roughness proxy, inspired by the International Roughness Index, normalises total acceleration by speed to isolate roughness-induced acceleration:

$$\text{rough_proxy} = \frac{\|\mathbf{a}\|}{v_{\text{kmh}} + 1} \quad (4)$$

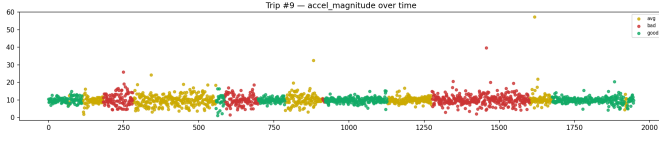


Fig. 4. Timeline of the 13 data collection trips showing sensor activity periods. Each horizontal band is one trip; colours indicate the labelled road condition (green = good, yellow = average, red = bad, purple = obstruction).

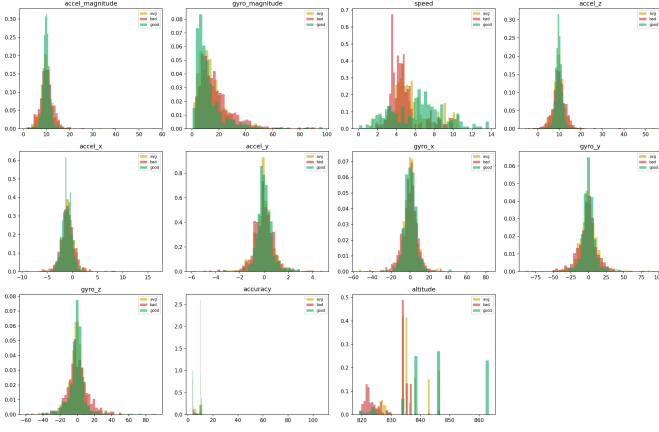


Fig. 5. Distribution of all 22 engineered features for each road condition class. Features such as `rough_proxy` and `gyro_magnitude` show the strongest class separation.

Compass heading θ is encoded as a point on the unit circle to eliminate the $0/360^\circ$ discontinuity:

$$\text{heading_sin} = \sin\left(\frac{\pi\theta}{180}\right), \quad \text{heading_cos} = \cos\left(\frac{\pi\theta}{180}\right) \quad (5)$$

Speed is converted to km/h and discretised into four ordinal bins (≤ 10 , $10-30$, $30-60$, > 60 km/h) as `speed_bucket`. The complete feature vector $\mathbf{x}$ is 22-dimensional, combining raw sensor values with all the engineered features described above.

V. EXPERIMENTAL RESULTS

A. Experimental Setup

The 1,948 three-class samples are split 80/20 (stratified, `random_state=42`) into 1,558 training and 390 test samples. Missing values from transient GPS fix losses are imputed with per-feature training-set medians; a `StandardScaler` is applied to scale-sensitive classifiers (LR, SVM, KNN, MLP) while tree-based models receive unscaled features. Six classifiers spanning a spectrum of complexity are trained and evaluated on accuracy, weighted precision/recall/F1, ROC-AUC, 5-fold stratified cross-validation F1, training time, and per-sample inference latency.

B. Classifier Comparison

Table III shows that XGBoost achieves the best result on every single metric. Of 390 test samples only 10 are misclassified (2.56% error rate), and the ROC-AUC of 0.9991

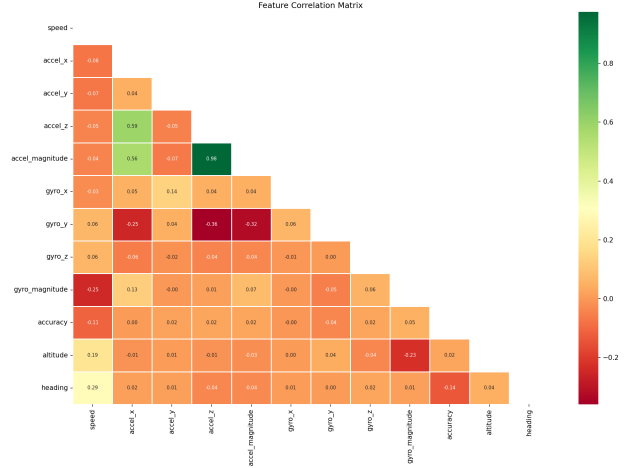


Fig. 6. Pearson correlation heatmap of the 22 engineered features. Speed-derived features form a strongly correlated cluster; acceleration and gyroscope features contribute complementary information.

implies essentially perfect class-probability rank ordering. The 5-fold CV F1 of 0.9512 ± 0.009 is slightly below the test F1—which is the expected direction—with a low standard deviation confirming performance stability. XGBoost trains in 0.898 s on commodity hardware and predicts at 0.004 ms per sample, meaning the FastAPI endpoint can theoretically serve over 250,000 predictions per second in single-threaded mode.

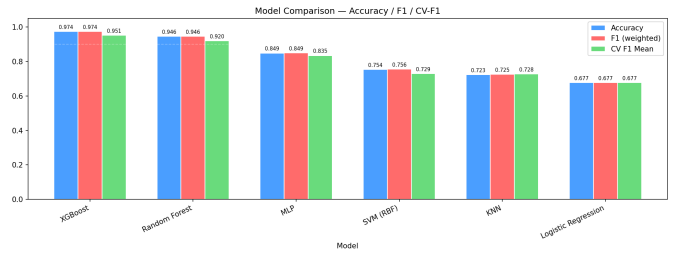


Fig. 7. Grouped bar chart of accuracy, weighted F1, and ROC-AUC across all six classifiers. XGBoost and Random Forest dominate by a substantial margin.

C. XGBoost Per-Class Performance

Table IV shows per-class results for the best-performing model. The confusion matrices confirm that XGBoost’s 10 errors are all adjacent-class confusions (average–good or bad–average); no good sample is ever classified as bad or vice versa—which is precisely the most safety-critical error type you’d want to avoid. The bad class achieves a perfect F1-score of 1.000, ensuring every genuinely bad segment gets flagged for repair.

D. Feature Importance Analysis

Feature importance analysis reveals that altitude (gain = 0.216) is the single most important feature, reflecting the geographic structure of the collection area where higher-elevation arterials tend to be better maintained. Heading cir-

TABLE III
CLASSIFIER PERFORMANCE COMPARISON (TEST SET: 390 SAMPLES, TRAINING SET: 1,558 SAMPLES)

Model	Accuracy	Precision	Recall	F1 (W)	ROC-AUC	CV F1 (mean $\pm$ std)	Train (s)	Infer (ms/samp.)
XGBoost	0.9744	0.9746	0.9744	0.9743	0.9991	0.9512 $\pm$ 0.009	0.898	0.0039
Random Forest	0.9462	0.9467	0.9462	0.9463	0.9961	0.9204 $\pm$ 0.024	0.525	0.0250
MLP	0.8487	0.8489	0.8487	0.8488	0.9507	0.8345 $\pm$ 0.019	1.836	0.0031
SVM (RBF)	0.7538	0.7778	0.7538	0.7558	0.8985	0.7288 $\pm$ 0.035	0.205	0.0334
KNN	0.7231	0.7329	0.7231	0.7253	0.8977	0.7278 $\pm$ 0.027	0.000	0.1290
Logistic Regression	0.6769	0.6785	0.6769	0.6769	0.8551	0.6767 $\pm$ 0.026	0.012	0.0005

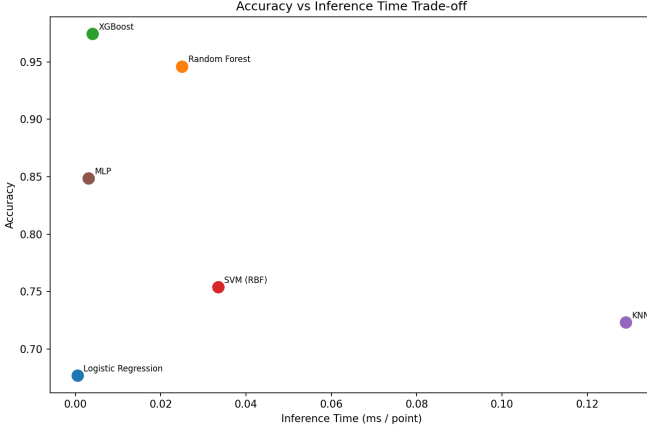


Fig. 8. Accuracy vs. inference latency trade-off. XGBoost achieves the highest accuracy and one of the lowest latencies (0.004 ms/sample). KNN is the clear outlier at 0.129 ms/sample.

TABLE IV
XGBOOST PER-CLASS CLASSIFICATION REPORT (TEST SET)

Class	Precision	Recall	F1-Score	Support
Average	0.972	0.972	0.972	142
Bad	1.000	1.000	1.000	109
Good	0.957	0.957	0.957	139
Macro Avg	0.977	0.977	0.977	390
Weighted Avg	0.974	0.974	0.974	390

cular encoding contributes 22.4% of total importance, capturing direction-dependent maintenance jurisdiction patterns. GPS accuracy (gain = 0.109) encodes indirect environmental information, as poor fix quality tends to co-occur with narrow, less-maintained roads. The roughness proxy and speed features validate key engineering decisions; low individual gyroscope axis importance relative to the derived magnitude confirms the value of using orientation-invariant norms rather than raw axis values.

E. Comparison with Related Work

Table V compares SENSRO with prior work on this kind of problem. SENSRO's XGBoost classifier achieves the highest accuracy of any of the compared systems, including the deep learning approach of Varona et al. [4]. Direct cross-system comparison is inherently limited—each system is evaluated on its own dataset—but the results do confirm that principled feature engineering with gradient boosting is competitive with or even superior to more complex architectures at this dataset

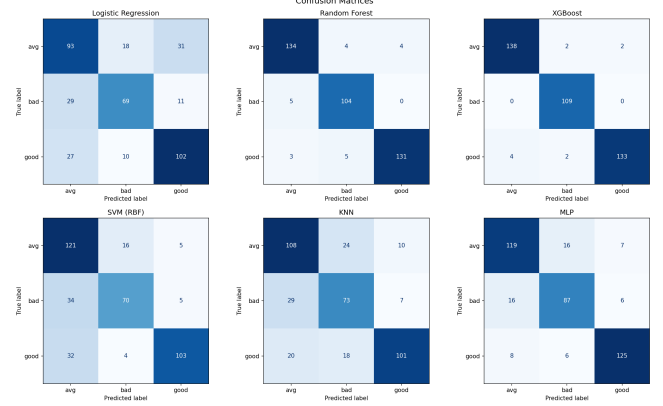


Fig. 9. Normalised confusion matrices for all six classifiers. XGBoost (top left) is near-diagonal; Logistic Regression (bottom right) shows systematic confusion between all class pairs.

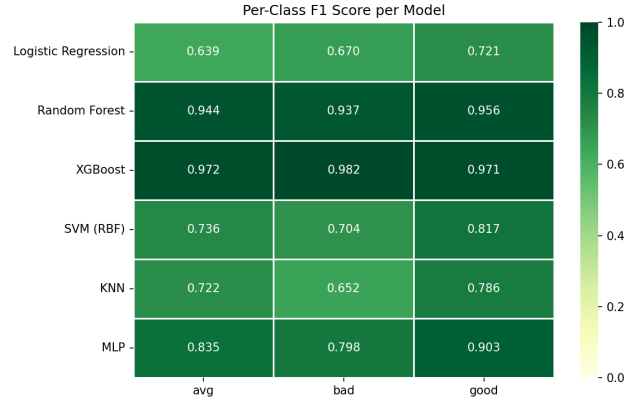


Fig. 10. Per-class F1-score for each classifier. XGBoost achieves F1 > 0.97 for all three classes; the bad class achieves perfect F1 = 1.000.

scale. XGBoost outperforms MLP by 12.6 percentage points, which is consistent with the broader tabular ML literature finding that tree methods generally dominate on small-to-medium tabular datasets where neural networks simply lack enough data to learn robust internal representations.

VI. DISCUSSION

A. Effectiveness of the Feature Engineering Pipeline

The 22-feature engineered representation provides substantially stronger class separation than raw axis readings alone. Orientation-invariant norms—Euclidean magnitudes across ac-

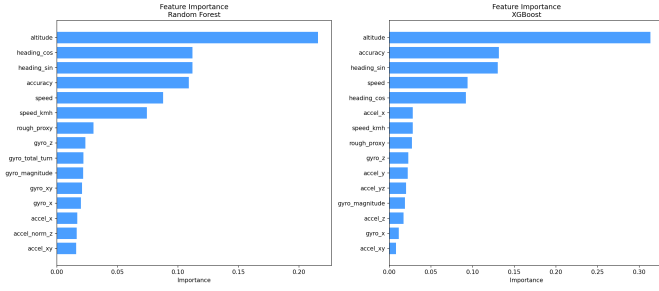


Fig. 11. XGBoost feature importance (mean gain). Altitude, heading circular encoding, and GPS accuracy are most discriminative, followed by speed and the roughness proxy.

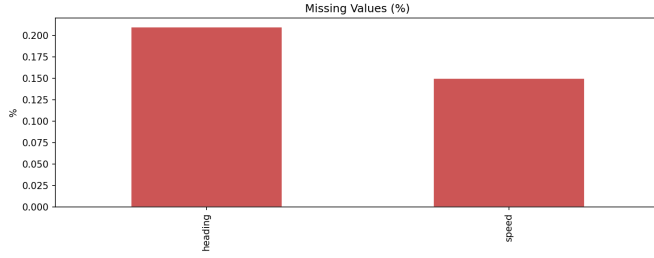


Fig. 12. Missing value profile across all 22 features. Speed-related features show slightly elevated missingness ($\approx 1.2\text{--}2.8\%$) from transient GPS fix losses; all are imputed with training-set medians.

celerometer and gyroscope axes—eliminate the need for a fixed device mounting and generalise reasonably well across different vehicles. The roughness proxy directly mirrors the International Roughness Index concept and, combined with speed binning, decouples road roughness from driver behaviour quite effectively. The circular encoding of heading prevents the $0/360^\circ$ wrap-around discontinuity that would otherwise corrupt gradient-based learners. Together, these design choices go a long way toward explaining why XGBoost, given well-engineered tabular features, outperforms the MLP by over 12 percentage points.

B. Why XGBoost Outperforms Other Classifiers

XGBoost’s dominance here is consistent with the broader tabular ML literature. With only 1,558 training samples, neural networks and SVMs lack either sufficient data or appropriate inductive biases to match gradient-boosted trees. Random Forest is the closest competitor (94.62%), confirming that ensemble tree methods suit this kind of problem well. Logistic

Regression’s relatively poor performance (67.69%) confirms that the class boundaries in this feature space are decidedly non-linear.

C. Limitations and Future Work

The results do carry some important caveats worth being upfront about. All 13 trips were conducted in a $\approx 1.2\text{ km}^2$ area of northern Bengaluru over just two days in May 2026 (pre-monsoon), using a single vehicle and driver. The model may have learned GPS-coordinate-specific altitude and heading patterns that simply won’t generalise well to other parts of the city, other Indian cities, or post-monsoon road conditions. Road condition labels were assigned by a single co-pilot using perceptual judgment, introducing subjectivity—especially for the sometimes-ambiguous average class. With 1,948 training samples, statistical power is limited; the gap between CV F1 (0.9512) and test F1 (0.9743) suggests there may be some mildly optimistic test-split luck at play.

Future work should address these limitations through multi-city data collection covering diverse road types and seasons, windowed feature extraction (sliding 1–2 s windows for spectral features), multi-vehicle and multi-driver validation, on-device ONNX inference to eliminate server dependency, crowdsourced Bayesian confidence aggregation across multiple users, and eventually integration with municipal GIS road management systems.

VII. CONCLUSION

This paper presented SENSRO, an end-to-end smartphone-based road condition detection and geo-referenced mapping system that can be deployed from any modern browser. Using a 22-feature engineered representation of accelerometer, gyroscope, GPS, and speed data from 1,948 labelled Bengaluru road samples, XGBoost achieves 97.44% accuracy, weighted F1 of 0.9743, ROC-AUC of 0.9991, and 0.004 ms/sample inference latency—with perfect detection of the safety-critical bad class. The browser-based PWA eliminates installation barriers entirely, making crowdsourced city-scale road monitoring practically accessible to any smartphone user. Our results demonstrate that principled feature engineering combined with gradient boosting offers a practical, low-cost, and scalable alternative to expensive conventional road survey equipment, effectively turning an everyday smartphone into a reliable road condition sensor.

REFERENCES

- [1] J. Eriksson, L. Girod, B. Hull, R. Newton, S. Madden, and H. Balakrishnan, “The pothole patrol: Using a mobile sensor network for road surface monitoring,” in *Proc. 6th Int. Conf. Mobile Systems, Applications, and Services (MobiSys)*, Breckenridge, CO, USA, Jun. 2008, pp. 29–39.
- [2] P. Mohan, V. N. Padmanabhan, and R. Ramjee, “Nericell: Rich monitoring of road and traffic conditions using smartphones,” in *Proc. 6th ACM Conf. Embedded Networked Sensor Systems (SenSys)*, Raleigh, NC, USA, Nov. 2008, pp. 323–336.
- [3] A. Mednis, G. Strazdins, R. Zviedris, G. Kanonirs, and L. Selavo, “Real time pothole detection using Android smartphones with accelerometers,” in *Proc. Int. Conf. Distributed Computing in Sensor Systems and Workshops (DCOSS)*, Barcelona, Spain, Jun. 2011, pp. 1–6.

TABLE V
COMPARISON WITH RELATED WORK

System	Classifier	Accuracy	Context
Pothole Patrol [1]	Threshold	$\approx 81\%$	US potholes
Nericell [2]	Threshold	$\approx 77\%$	India, multi-modal
Mednis et al. [3]	Threshold	90%+	Latvia, controlled
RoadMon [5]	Random Forest	88.4%	Lab dataset
Varona et al. [4]	1D-CNN	94.2%	Argentina
Proposed	XGBoost	97.44%	India, real-world

- [4] B. Varona, A. Monteserin, and A. Teyseyre, "A deep learning approach to automatic road surface monitoring and pothole detection," *Personal and Ubiquitous Computing*, vol. 24, no. 4, pp. 519–534, Aug. 2020.
- [5] A. Allouch, A. Koubaa, T. Abbes, and A. Ammar, "RoadMon: A smartphone-based road and traffic monitoring companion app," *Procedia Computer Science*, vol. 110, pp. 69–78, 2017.
- [6] W. Xu, R. Zhao, Y. Gao, and F. Cao, "Road surface condition classification using deep learning," in *Proc. IEEE Int. Conf. Image Processing (ICIP)*, Athens, Greece, Oct. 2018, pp. 3912–3916.
- [7] S. Ramírez, "FastAPI," 2018. [Online]. Available: <https://fastapi.tiangolo.com>
- [8] A. Boraskar, N. Vankadhara, B. Raman, and P. Kulkarni, "Wolverine: Traffic and road condition estimation using smartphone sensors," in *Proc. IEEE Int. Conf. Communication Systems and Networks (COM-SNETS)*, Bengaluru, India, Jan. 2012, pp. 1–6.
- [9] C. Christodoulou, A. Theodorides, and C. Pattichis, "Road condition monitoring using Android smartphones," in *Proc. IEEE 19th Int. Conf. Telecommunications (ICT)*, Jounieh, Lebanon, Apr. 2012, pp. 1–5.
- [10] H. Maeda, Y. Sekimoto, T. Seto, T. Kashiwayama, and H. Omata, "Road damage detection and classification using deep neural networks with smartphone images," *Computer-Aided Civil and Infrastructure Engineering*, vol. 33, no. 12, pp. 1127–1141, Dec. 2018.
- [11] A. Perttunen, O. Mazhelis, F. Cong, M. Riekkki, and J. Riekkki, "Distributed road surface condition monitoring using mobile phones," in *Proc. Int. Conf. Ubiquitous Intelligence and Computing (UIC)*, Banff, Canada, Sep. 2011, pp. 64–78.
- [12] M. Fazeen, B. Gozick, R. Dantu, M. Bhukhiya, and M. C. Gonzalez, "Safe driving using mobile phones," *IEEE Transactions on Intelligent Transportation Systems*, vol. 13, no. 3, pp. 1462–1468, Sep. 2012.
- [13] R. Kalim, J.-Y. Jeong, and S.-H. Lee, "CRATER: A crowdsourcing-based road condition monitoring system," in *Proc. IEEE Int. Conf. Ubiquitous and Future Networks (ICUFN)*, Shanghai, China, Jul. 2014, pp. 234–239.
- [14] K. Agrawal, A. Patel, S. Bhavnani, and P. Bhavsar, "Road quality assessment using smartphone sensors and machine learning," in *Proc. IEEE Int. Conf. Machine Learning and Applications (ICMLA)*, Boca Raton, FL, USA, Dec. 2019, pp. 1464–1469.
- [15] D. Strukov, A. Klimenko, and N. Filatov, "Machine learning methods for road surface quality classification using smartphone accelerometer data," *Applied Sciences*, vol. 12, no. 3, p. 1197, Jan. 2022.
- [16] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.
- [17] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, Aug. 2016, pp. 785–794.
- [18] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [19] S. Nienaber, M. J. Booysen, and R. S. Kroon, "Detecting road anomalies using a smartphone and the sensor fusion approach," in *Proc. 21st Intelligent Transport Systems World Congress (ITS)*, Detroit, MI, USA, Sep. 2014.
- [20] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, Jun. 2002.
- [21] C. Elkan, "The foundations of cost-sensitive learning," in *Proc. 17th Int. Joint Conf. Artificial Intelligence (IJCAI)*, Seattle, WA, USA, Aug. 2001, pp. 973–978.
- [22] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, Mar. 2003.
- [23] M. A. Hall, "Correlation-based feature selection for machine learning," Ph.D. dissertation, Dept. Computer Science, Univ. Waikato, Hamilton, New Zealand, 1999.
- [24] A. Russell, "Progressive web apps: The future of mobile development," *IEEE Software*, vol. 36, no. 1, pp. 44–52, Jan. 2019.
- [25] World Wide Web Consortium, "Generic Sensor API," W3C Candidate Recommendation, Jul. 2019. [Online]. Available: <https://www.w3.org/TR/generic-sensor/>
- [26] D. Potts, J. Walker, and A. Langford, "Crowdsourced road quality mapping for urban infrastructure management," *Transportation Research Part C: Emerging Technologies*, vol. 94, pp. 244–258, Sep. 2018.